

Deep Learning Spam Email Detection Using Neural Network

1. Objective

The objective of this project is to develop a Deep Learning model that can classify messages as Spam or Ham (Not Spam). The model is trained using a Neural Network and evaluated based on its prediction accuracy.

2. Dataset Description

The dataset contains spam and ham messages used for text classification. A total of 200 records were used in this project, consisting of 100 spam messages and 100 ham messages.

... (200, 2)

	text	label	
0	Win a free iPhone 0	spam	
1	Claim your cash prize 0	spam	
2	Get free recharge now 0	spam	
3	Exclusive offer click here 0	spam	
4	You won 10000 rupees 0	spam	

	text	label	
0	Win a free iPhone 0	1	
1	Claim your cash prize 0	1	
2	Get free recharge now 0	1	
3	Exclusive offer click here 0	1	
4	You won 10000 rupees 0	1	

3. Tools Used

- Google Colab
- Python
- TensorFlow
- Scikit-learn

4. Libraries Used

- Pandas
- TensorFlow / Keras
- Scikit-learn

5. Data Preprocessing

The dataset was prepared for Deep Learning by converting categorical labels into numerical values. Spam messages were assigned the value 1 and Ham messages were assigned the value 0.

6. Tokenization

Tokenization was performed to convert text messages into numerical sequences. This process helps the Neural Network understand textual information in a numerical format.

7. Padding

Padding was applied to ensure that all input sequences have the same length. This is required for Neural Network processing.

8. Train-Test Split

The dataset was divided into training and testing sets using an 80:20 ratio. The training set was used for learning and the testing set was used for evaluation.

9. Model Architecture

A Sequential Neural Network model was created using the following layers:

- Embedding Layer
- Flatten Layer
- Dense Hidden Layer
- Output Layer with Sigmoid Activation

... Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	?	0 (unbuilt)
flatten_3 (Flatten)	?	0 (unbuilt)
dense_6 (Dense)	?	0 (unbuilt)
dense_7 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

10. Model Training

The Neural Network was trained using the Adam optimizer and Binary Cross Entropy loss function. The model was trained for 10 epochs.

```
... Epoch 1/10
5/5 ----- 1s 64ms/step - accuracy: 0.7375 - loss: 0.6789 - val_accuracy: 0.7000 - val_loss: 0.6722
Epoch 2/10
5/5 ----- 0s 18ms/step - accuracy: 0.9312 - loss: 0.6522 - val_accuracy: 0.9500 - val_loss: 0.6509
Epoch 3/10
5/5 ----- 0s 19ms/step - accuracy: 0.9812 - loss: 0.6235 - val_accuracy: 1.0000 - val_loss: 0.6259
Epoch 4/10
5/5 ----- 0s 28ms/step - accuracy: 1.0000 - loss: 0.5900 - val_accuracy: 1.0000 - val_loss: 0.5938
Epoch 5/10
5/5 ----- 0s 19ms/step - accuracy: 1.0000 - loss: 0.5505 - val_accuracy: 1.0000 - val_loss: 0.5562
Epoch 6/10
5/5 ----- 0s 19ms/step - accuracy: 1.0000 - loss: 0.5060 - val_accuracy: 1.0000 - val_loss: 0.5138
Epoch 7/10
5/5 ----- 0s 20ms/step - accuracy: 1.0000 - loss: 0.4563 - val_accuracy: 1.0000 - val_loss: 0.4664
Epoch 8/10
5/5 ----- 0s 20ms/step - accuracy: 1.0000 - loss: 0.4023 - val_accuracy: 1.0000 - val_loss: 0.4143
Epoch 9/10
5/5 ----- 0s 18ms/step - accuracy: 1.0000 - loss: 0.3461 - val_accuracy: 1.0000 - val_loss: 0.3606
Epoch 10/10
5/5 ----- 0s 20ms/step - accuracy: 1.0000 - loss: 0.2914 - val_accuracy: 1.0000 - val_loss: 0.3084
```

11. Accuracy

The trained model was evaluated using the test dataset. The model achieved an accuracy of 100%.

```
... 2/2 ————— 0s 23ms/step - accuracy: 1.0000 - loss: 0.3084
```

```
Accuracy: 1.0
```

Accuracy Obtained: **1.0 (100%)**

12. Conclusion

In this project, a Deep Learning-based Spam Email Detection system was successfully developed using TensorFlow and Keras. Text preprocessing techniques such as tokenization and padding were applied before training the Neural Network. The model achieved high accuracy and successfully classified spam and ham messages. This project demonstrates the effectiveness of Deep Learning techniques in text classification task.